

HNS

Human Natural Structure Implementation Specification

Version 1.0

May 2026

Structural OS for Human Understanding

satoru-hara / 03_NSW

Table of Contents

- 0. Preface 6
 - 0.1 Purpose of This Specification 6
 - 0.2 Intended Audience 7
 - 0.3 Relationship to Other HNS Documents 7
 - 0.4 Document Conventions and Terminology 9
- 1. Introduction 10
 - 1.1 Background and Motivation 10
 - 1.2 The Role of HNS as a Structural Operating System 11
 - 1.3 Scope of This Specification 12
 - 1.4 Non-Goals and Out-of-Scope Items 12
- 2. System Overview 14
 - 2.1 Conceptual Architecture of HNS 14
 - 2.2 Core Principles of Structural OS Design 15
 - Principle 1: Causal Priority 15
 - Principle 2: Structural Persistence 15
 - Principle 3: Interpretive Charity 15
 - Principle 4: Verifiable Transparency 15
 - Principle 5: Minimal Footprint 16
 - 2.3 Interaction Model Between HNS and AI Systems 16
 - 2.4 Relationship to HNS Foundational / Observational / Analytical Specifications 17
- 3. HNS Core Architecture 18
 - 3.1 HNS Core Engine 18
 - 3.1.1 Input Processing Pipeline 18
 - 3.1.2 HNS Coherence Constraints 19
 - 3.2 Structural Coordinate System (HNS-36) 21
 - 3.2.1 Structural Axes 22
 - 3.2.2 Layer Definitions 23
 - Layer 1: Physical (L1) 24
 - Layer 2: Perceptual (L2) 24
 - Layer 3: Cognitive (L3) 24
 - Layer 4: Intentional (L4) 24
 - Layer 5: Expressive (L5) 25
 - Layer 6: Reflexive (L6) 25
 - 3.2.3 HNS-36 Vector Representation 25
 - 3.2.4 Coordinate Initialization 25
 - 3.2.5 Coordinate Update Rules 26
 - 3.3 Human Natural Layers (6-Layer Causal Stack) 26
 - 3.4 Structural Feedback Engine (HSF) 27
 - 3.4.1 Structural Feedback Signal Format 27

- 3.5 External Verification Architecture (EVA) Integration Points28
- 4. Data Model and Representations31
 - 4.1 HNS Structural Entities31
 - 4.1.1 StructuralSession31
 - 4.1.2 StructuralTurn32
 - 4.2 Context, Role, Intention, Constraint Models34
 - 4.2.1 Context Model34
 - 4.2.2 Role Model34
 - 4.2.3 Intention Model35
 - 4.2.4 Constraint Model35
 - 4.3 JSON / JSON-LD Schema Definitions35
 - 4.3.1 HNS-36 Coordinate JSON-LD Schema35
 - 4.3.2 StructuralSnapshot JSON-LD Schema36
 - 4.4 RDF / Knowledge Graph Mapping37
 - 4.5 Session, Turn, and Structural Snapshot Formats39
- 5. APIs and Interfaces40
 - 5.1 API Overview40
 - 5.2 Core Endpoints40
 - 5.2.1 POST /hns/analyze40
 - Request Body40
 - Response Body (200 OK)41
 - Error Codes41
 - 5.2.2 POST /hns/structure42
 - Request Body42
 - Response Body (200 OK)43
 - 5.2.3 POST /hns/feedback43
 - Request Body43
 - Response Body (200 OK)43
 - 5.2.4 POST /hns/evaluate (HNS-36)44
 - Request Body44
 - Response Body (200 OK)44
 - 5.3 Session Management Endpoints45
 - 5.3.1 POST /hns/session/start45
 - 5.3.2 GET /hns/session/{sessionId}45
 - 5.3.3 POST /hns/session/{sessionId}/end45
 - 5.3.4 GET /hns/session/{sessionId}/turns45
 - 5.4 Input/Output Specifications45
 - 5.5 Error Handling and Fallback Behavior46
 - 5.6 Logging and Traceability Requirements46
- 6. Integration Patterns48
 - 6.1 Embedding HNS in LLM Pipelines48

6.1.1 Structural Context Prompt Template	48
6.2 Middleware Integration for Dialogue Systems	49
6.3 HNS as a Structural OS Layer in Multi-Agent Systems ...	49
6.4 Integration with EVA for Compliance and Safety	50
6.5 Deployment Models (Local, Cloud, Hybrid)	50
Local Deployment	50
Cloud Deployment	50
Hybrid Deployment	51
7. Minimal Viable Implementation (MVI)	52
7.1 Overview	52
7.2 Required Components	52
7.3 Optional Components	53
7.4 Minimal Data Requirements	54
7.5 Minimal Evaluation Requirements	55
7.6 Example Implementation Flow	55
8. Use Cases and Reference Workflows	57
8.1 Structural Interpretation of User Intent	57
8.2 Long-Horizon Dialogue Consistency	57
8.3 Safety and Governance Applications	58
8.4 Multi-Agent Coordination	58
8.5 Human-AI Interaction Enhancement	59
9. Conformance Requirements	60
9.1 Mandatory Requirements	60
9.2 Recommended Requirements	61
9.3 Optional Extensions	62
9.4 Conformance Levels (A / AA / AAA)	63
9.5 Test Cases and Validation Procedures	63
10. Security, Safety, and Governance	65
10.1 Structural Safety Principles	65
10.1.1 Structural Inertia Principle	65
10.1.2 Goal Boundary Enforcement	66
10.2 EVA Integration Requirements	66
10.3 Logging and Auditability	67
10.4 Privacy and Data Handling	67
10.5 Risk Management Framework	68
11. Performance and Evaluation	70
11.1 Structural Accuracy Metrics	70
11.2 HNS-36 Evaluation Protocol	70
11.2.1 Evaluation Procedure	70
11.2.2 Weighted Evaluation	71
11.3 Latency and Throughput Considerations	71
11.4 Benchmarking Methodology	72

- 12. Licensing and Intellectual Property.....74
 - 12.1 HNS Licensing Model74
 - 12.1.1 Open Specification License (OSL).....74
 - 12.1.2 Commercial Implementation License (CIL).....74
 - 12.2 Structural OS Trademark and Naming Rules75
 - 12.3 Implementation Rights and Restrictions.....75
 - 12.4 Compliance with Open Standards.....76
- 13. Future Extensions78
 - 13.1 Planned Extensions to HNS.....78
 - HNS-72: Extended Coordinate System78
 - HNS-Train: Training Integration Protocol.....78
 - HNS-Multimodal: Multimodal Structural Analysis.....78
 - HNS-Collective: Multi-Party Structural Modeling79
 - 13.2 Interoperability with Global Standards.....79
 - 13.3 Evolution of the Structural OS Concept.....79
- Appendix A: Glossary.....81
- Appendix B: JSON-LD Examples85
 - B.1 Complete StructuralSession Example85
 - B.2 StructuralEvaluation Response Example.....86
- Appendix C: Reference Architecture Diagrams87
 - C.1 HNS System Architecture Overview87
 - C.2 HNS Turn Processing Sequence.....87
- Appendix D: Mapping to International Standards89
 - D.1 Mapping to ISO/IEC 42001:2023 (AI Management Systems)89
 - D.2 Mapping to EU AI Act (Regulation 2024/1689)90
- Appendix E: Change Log92
 - Version 1.0 (May 2026) — Initial Release.....92

0. Preface

0.1 Purpose of This Specification

The HNS Implementation Specification (HNS-IS) defines the normative requirements, recommended practices, and reference architectures for implementing the Human Natural Structure (HNS) framework in software systems, artificial intelligence pipelines, and dialogue platforms. It provides implementers, system architects, and standards bodies with the authoritative technical guidance needed to build conformant HNS-based systems.

HNS is a Structural Operating System for Human Understanding — a multi-layered, coordinate-based framework that models the causal architecture of human cognition, behavior, and meaning generation. Where prior AI integration frameworks focus on syntactic or statistical alignment, HNS introduces a structural coordinate layer that captures the causal relationships between a human agent's physical context, perceptual state, cognitive interpretation, intentional goals, expressive choices, and reflexive evaluation.

This specification is the primary implementation-facing document in the HNS document series. It translates the conceptual foundations established in the HNS Foundational Specification, the observational framework of the HNS Observational Specification, and the analytical methods of the HNS Analytical Specification into actionable engineering requirements. Implementers who build fully conformant systems based on this specification will produce HNS-enabled software that is interoperable with other conformant implementations and auditable against the External Verification Architecture (EVA).

This document has been authored to serve as both a standalone engineering reference and an internationally submittable technical standard. Its structure and terminology are aligned with ISO/IEC directives for technical specifications, and its conformance framework is compatible with the requirements of ISO/IEC JTC 1 / SC 42 and the requirements of the EU AI Act.

0.2 Intended Audience

This specification is intended for the following audiences:

- Software engineers and system architects implementing HNS as a component in AI systems, dialogue platforms, or human-computer interaction frameworks.
- AI researchers and data scientists designing evaluation protocols, training pipelines, or benchmarking methodologies that incorporate HNS structural coordinates.
- Standards bodies and regulatory authorities assessing HNS compliance requirements for AI safety and governance frameworks.
- Technical product managers and enterprise architects planning HNS adoption within organizational technology stacks.
- Integration partners and third-party vendors seeking to build HNS-compatible tooling, middleware, or evaluation services.

Readers are expected to have familiarity with software architecture principles, RESTful API design, JSON-LD and semantic data modeling, and basic concepts in natural language processing and human-computer interaction. Familiarity with the HNS Foundational Specification is strongly recommended but not strictly required; this specification includes sufficient definitional content to be used independently.

0.3 Relationship to Other HNS Documents

This specification is the seventh document in the HNS technical series. The following table summarizes the relationship between this document and the other publications in the series:

Book No.	Title	Relationship to This Spec
07	HNS Foundational	Defines the conceptual basis, axioms, and terminology used

Book No.	Title	Relationship to This Spec
	Specification	throughout this implementation spec.
08	HNS Observational Specification	Provides the observational categories and signal types that feed into HNS-36 coordinate computation.
09	HNS Analytical Specification	Defines the analytical methods for computing, validating, and interpreting HNS-36 coordinates.
10	HNS Operating System	Defines the OS-level behavior that this implementation spec operationalizes.
11	HNS Technical Note	Provides supplementary technical detail supporting select sections of this specification.
12	External Verification Architecture (EVA)	Defines the external audit and verification system referenced in Chapters 3, 6, and 10.
13	HNS Structural Feedback (HSF)	Defines the feedback engine whose API and integration model are specified in Chapter 3 and 5.
14	HNS Foundational Specification FO	Formal ontology layer; referenced in Chapter 4 for JSON-LD and RDF representations.
15	HNS Foundational Specification CS	Coordinate system formal specification; ground truth for Chapter 3 and Chapter 11 metrics.

When normative requirements in this document conflict with requirements in other HNS documents, this specification takes precedence for implementation matters. Conceptual and definitional conflicts should be resolved by reference to the HNS Foundational Specification (Book 07).

0.4 Document Conventions and Terminology

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in RFC 2119.

Throughout this document, the following typographic conventions apply:

- Monospaced font (e.g., `hns:layer1.activation`) denotes code, identifiers, field names, and API paths.
- Bold text denotes a defined term being introduced for the first time in a given section.
- Italic text denotes emphasis, foreign terms, or titles of other documents.
- Numbered requirements (R-XXX) denote normative requirements that appear in conformance tables.
- Notes in bordered boxes provide informative guidance and do not constitute normative requirements.

All version references in this specification refer to Version 1.0 of the respective documents unless otherwise noted. Version-specific differences are documented in Appendix E (Change Log).

1. Introduction

1.1 Background and Motivation

The rapid proliferation of large language models (LLMs) and AI dialogue systems has created a critical gap in the architecture of human-AI interaction: the absence of a principled structural layer between raw human input and AI processing. Current AI systems process human language as a statistical stream of tokens, with no explicit representation of the causal structure underlying human cognition — the layers of physical context, perceptual framing, cognitive reasoning, intentional goal-setting, expressive choice, and reflexive evaluation that constitute every meaningful human utterance.

This architectural gap produces well-documented failure modes: AI systems that respond accurately to surface-level linguistic patterns while failing to honor the deeper structural intent of the human; systems that cannot maintain consistent structural understanding across multi-turn dialogues; and systems that lack an auditable trail of structural reasoning that could be submitted to regulatory verification. The absence of a structural OS layer means that every AI interaction begins from scratch, with no persistent structural model of the interlocutor.

Human Natural Structure (HNS) addresses this gap by introducing a six-layer causal stack — the Human Natural Layers (HNL) — together with a 36-dimensional coordinate system (HNS-36) that encodes the structural state of a human agent at any given moment in an interaction. By maintaining and updating this structural representation across turns, HNS-enabled systems can provide structurally coherent, intentionally aligned, and verifiably safe AI responses.

The motivation for a formal implementation specification arises from the need to ensure that HNS is implemented consistently across different platforms, vendors, and deployment contexts. Without a normative specification, independent implementations of HNS would diverge in their interpretation of the coordinate system, feedback mechanisms, and evaluation protocols, making interoperability and independent verification impossible.

1.2 The Role of HNS as a Structural Operating System

The term Structural Operating System (Structural OS) is used deliberately and precisely in the HNS framework. Like a conventional operating system, HNS operates as an intermediary layer between the underlying substrate (human cognition and language) and the application layer (AI responses and services). It provides resource management, process coordination, and a stable interface that application-layer components can rely on without needing to reimplement low-level structural reasoning.

Specifically, HNS performs the following OS-level functions:

- **Structural Initialization:** When a new session begins, HNS initializes the HNS-36 coordinate space based on available context signals, establishing a baseline structural model of the human agent.
- **Structural Update:** At each turn, HNS updates the coordinate space based on new input, using the Structural Feedback Engine (HSF) to incorporate corrective signals when structural drift is detected.
- **Structural Scheduling:** HNS prioritizes which layers of the causal stack require most attention in generating a response, effectively scheduling cognitive resources across layers.
- **Structural Memory Management:** HNS maintains a session-scoped structural memory that persists across turns, enabling long-horizon coherence without the token-space overhead of full conversational replay.
- **Structural Verification Interface:** HNS exposes a standardized interface to the External Verification Architecture (EVA), enabling real-time and post-hoc auditing of structural behavior.

This OS framing positions HNS not as a competing model or an alternative to LLMs, but as a complementary structural layer that sits between the human interlocutor and the AI model. Any LLM or dialogue system can be augmented with HNS by implementing the APIs and data models defined in this specification.

1.3 Scope of This Specification

This specification defines the following:

1. The HNS Core Architecture, including the HNS Core Engine, HNS-36 coordinate system, Human Natural Layers, Structural Feedback Engine, and EVA integration points.
2. The HNS Data Model, including structural entities, JSON-LD schemas, RDF mappings, and session/turn formats.
3. The HNS API surface, including endpoint specifications, input/output formats, error handling, and logging requirements.
4. Integration patterns for embedding HNS in LLM pipelines, middleware, and multi-agent systems.
5. Minimal Viable Implementation (MVI) requirements for systems seeking basic conformance.
6. Reference use cases and workflows demonstrating HNS application.
7. Conformance requirements and test cases for verifying implementation correctness.
8. Security, safety, governance, performance, and licensing requirements.

This specification covers both server-side HNS deployments (where HNS is implemented as a service that other systems call) and embedded HNS deployments (where HNS is integrated directly into an AI system's inference pipeline). It covers both real-time and batch processing modes.

1.4 Non-Goals and Out-of-Scope Items

The following items are explicitly outside the scope of this specification:

- Training methodologies for LLMs using HNS structural feedback. This is addressed in separate HNS training specifications.
- Natural language parsing algorithms. HNS assumes that text input has already been tokenized or embedded; it

operates on the semantic and structural level above raw text.

- User interface design for HNS-enabled applications. HNS is a backend structural framework; UI design is application-specific.
- Specific hardware or cloud platform requirements. HNS is platform-agnostic; deployment infrastructure is the responsibility of the implementer.
- Psycholinguistic models of human language acquisition. HNS draws on cognitive science but is an engineering framework, not a scientific theory of mind.
- Legal advice on AI regulation compliance. This specification provides mappings to regulatory frameworks (e.g., EU AI Act) for technical alignment purposes only.

2. System Overview

2.1 Conceptual Architecture of HNS

At its highest level, the HNS architecture consists of five integrated components arranged in a layered topology. The following description corresponds to the reference architecture diagram provided in Appendix C.

The Human Input Layer receives raw input from a human interlocutor in any supported modality (text, voice transcript, structured data, or multimodal input). This layer performs preprocessing, normalization, and initial signal extraction, producing a structured input representation that is passed to the HNS Core Engine.

The HNS Core Engine is the central processing unit of the architecture. It receives preprocessed input, accesses the current session's structural state from the Structural Memory Store, and orchestrates the analysis of the input across all six Human Natural Layers. The Core Engine produces an updated HNS-36 coordinate set and, where applicable, a Structural Feedback Signal (SFS) for the active session.

The Structural Memory Store is a persistent, session-scoped data store that maintains the HNS-36 coordinate history, structural snapshots, and feedback records for each active session. It supports both real-time queries from the Core Engine and batch retrieval for analysis and auditing.

The AI Response Layer receives the updated HNS-36 coordinate set from the Core Engine, along with structural directives derived from the current session state. These coordinates and directives inform the generation of structurally aligned AI responses. The AI Response Layer may be implemented by any LLM or dialogue system; HNS provides the structural grounding; the LLM provides the linguistic surface.

The External Verification Architecture (EVA) interface is an asynchronous, append-only logging system that captures all structural state transitions, feedback events, and evaluation scores for external audit. EVA operates independently of the main processing pipeline and is designed to be tamper-evident

and regulatorily submittable.

2.2 Core Principles of Structural OS Design

The following principles govern all design decisions in HNS and in this implementation specification:

Principle 1: Causal Priority

HNS models human behavior as causally structured, not statistically correlated. The six Human Natural Layers are ordered by causal priority: each layer provides the necessary structural conditions for the layer above it. Physical conditions constrain and enable perceptual states; perceptual states constrain and enable cognitive interpretations; and so on. This causal ordering is reflected in the coordinate update rules specified in Chapter 3 and must be honored by all conformant implementations.

Principle 2: Structural Persistence

HNS structural coordinates persist across turns within a session. Unlike stateless AI systems that treat each turn as an independent event, HNS maintains a continuously updated structural model of the interlocutor. This persistence is the basis for long-horizon coherence, progressive structural refinement, and meaningful structural feedback. Implementations **MUST** maintain structural state across all turns within a session.

Principle 3: Interpretive Charity

When structural signals are ambiguous or underspecified, HNS resolves ambiguity in favor of the interpretation that best preserves the coherence of the causal stack. This is formalized in the HNS Charitable Interpretation Rule (CIR), which specifies that when multiple coordinate assignments are compatible with the observed input, the assignment that minimizes structural discontinuity with the prior session state **SHALL** be preferred.

Principle 4: Verifiable Transparency

All structural decisions — coordinate assignments, feedback applications, evaluation scores — **MUST** be logged in a format

that is machine-readable, auditable, and suitable for submission to regulatory verification processes. This principle is operationalized through the EVA integration requirements specified in Chapter 10.

Principle 5: Minimal Footprint

HNS is designed to augment existing AI systems with minimal intrusion. The HNS-36 coordinate set is compact (36 floating-point values), the API surface is small and RESTful, and the data model uses standard formats (JSON-LD, RDF) that integrate naturally with existing semantic web infrastructure. Implementations **SHOULD** minimize computational overhead, targeting latency budgets specified in Chapter 11.

2.3 Interaction Model Between HNS and AI Systems

The canonical HNS interaction model operates on a per-turn basis within a session lifecycle. The following sequence describes the normative interaction flow:

9. **Session Initialization:** The AI system calls `POST /hns/session/start` with session metadata. HNS initializes a new structural state with all HNS-36 coordinates at their prior distribution values or at implementation-defined defaults.
10. **Turn Input Reception:** The human interlocutor provides input. The AI system passes this input (along with turn metadata) to `POST /hns/analyze`.
11. **Structural Analysis:** The HNS Core Engine analyzes the input across all six Human Natural Layers, computing updated HNS-36 coordinates and generating a structural directive.
12. **Response Generation:** The AI system uses the returned HNS-36 coordinates and structural directive to generate a structurally aligned response.
13. **Structural Feedback:** If the AI system detects a structural misalignment in its own response, it **MAY** call `POST /hns/feedback` with a corrective signal. EVA logs this event.

14. Turn Closure: The AI system calls POST /hns/structure to store the completed turn's structural snapshot. The session state is updated.
15. Session Termination: When the session ends, the AI system calls POST /hns/session/end. EVA receives the complete session structural log.

2.4 Relationship to HNS Foundational / Observational / Analytical Specifications

This implementation specification operationalizes the concepts, observations, and analytical methods defined in the three preceding HNS specifications. The Foundational Specification (Book 07) establishes the axioms and definitions that this specification treats as given; specifically, the definitions of the six Human Natural Layers, the HNS-36 coordinate schema, and the Structural OS concept. Implementers encountering undefined terms should refer to Book 07.

The Observational Specification (Book 08) defines the observational categories — physical signals, behavioral markers, linguistic features, and contextual indicators — that are the inputs to the HNS-36 coordinate computation process. The HNS Core Engine described in Chapter 3 of this specification consumes these observational categories as part of its input analysis pipeline. The exact mapping from observational categories to coordinate values is specified in this document (Chapter 3) and supplemented by the Analytical Specification (Book 09).

The Analytical Specification (Book 09) defines the mathematical methods for computing, updating, and evaluating HNS-36 coordinates. This implementation specification references those methods by name and specifies the API contracts that implementations must expose; the underlying mathematical detail is contained in Book 09 and is not reproduced here.

3. HNS Core Architecture

3.1 HNS Core Engine

The HNS Core Engine (HCE) is the central processing component of an HNS implementation. It is responsible for receiving preprocessed input representations, orchestrating structural analysis across the six Human Natural Layers, computing HNS-36 coordinate updates, generating structural directives, and producing Structural Feedback Signals where applicable.

The HCE must be implemented as a stateless processing component that reads structural state from the Structural Memory Store at the beginning of each turn and writes updated state at the end of each turn. The HCE itself SHALL NOT maintain in-process state between API calls; all persistent state is the responsibility of the Structural Memory Store.

The HCE exposes its functionality exclusively through the HNS API defined in Chapter 5. Implementations MUST NOT expose internal HCE data structures or processing state through the API surface; all outputs must be expressed in terms of the data model defined in Chapter 4.

3.1.1 Input Processing Pipeline

The HCE input processing pipeline consists of the following sequential stages, each of which MUST be implemented in conformant systems:

16. Signal Extraction: Raw input (text, transcribed audio, or structured data) is analyzed to extract observational signals as defined in the HNS Observational Specification. Signals are categorized by their associated Human Natural Layer and represented as floating-point activation values in the range [0.0, 1.0].
17. Context Retrieval: The current session's HNS-36 coordinate set is retrieved from the Structural Memory Store. If no prior coordinates exist (first turn), implementation-defined defaults are used.

- 18. **Coordinate Computation:** The extracted signals are combined with the prior coordinate set using the coordinate update rules defined in Section 3.2.5. The result is a proposed updated HNS-36 coordinate set.
- 19. **Coherence Validation:** The proposed coordinate set is validated against the HNS Coherence Constraints defined in Section 3.1.2. If constraints are violated, the coordinate set is adjusted using the Structural Repair Algorithm (SRA) defined in Book 09.
- 20. **Directive Generation:** Based on the validated coordinate set, the HCE generates a Structural Directive — a compact summary of the structural state that informs AI response generation.
- 21. **State Commit:** The validated coordinate set and all processing metadata are written to the Structural Memory Store as a new StructuralSnapshot.

3.1.2 HNS Coherence Constraints

The following coherence constraints apply to all HNS-36 coordinate sets. Conformant implementations MUST enforce these constraints; any proposed coordinate set that violates a constraint must be repaired before being committed.

Constraint ID	Name	Formal Statement	Severity
CC-01	Layer Monotonicity	For any two adjacent layers Li and $Li+1$, if $Activation(Li) < 0.1$, then $Activation(Li+1)$ MUST be < 0.2 .	REQUIRED
CC-02	Integration Symmetry	The Integration axis value for	REQUIRED

Constraint ID	Name	Formal Statement	Severity
		Li with respect to Li+1 MUST equal the Integration axis value for Li+1 with respect to Li, within tolerance ± 0.05 .	
CC-03	Intentional Coherence	If the Intentional Layer (L4) Goal axis increases by more than 0.3 in a single turn, the Cognitive Layer (L3) Reasoning axis MUST also increase by at least 0.1.	REQUIRED
CC-04	Reflexive Dampening	The Reflexive Layer (L6) Adaptation axis MUST NOT decrease by more than 0.4 in a single turn without a corresponding Structural	REQUIRED

Constraint ID	Name	Formal Statement	Severity
		Feedback Signal.	
CC-05	Physical Grounding	The Physical Layer (L1) mean activation MUST be within the range [0.05, 0.98] for all non-null inputs.	REQUIRED
CC-06	Expressive Consistency	The Expressive Layer (L5) Style axis MUST NOT change by more than 0.5 between consecutive turns without an explicit context-shift signal.	RECOMMENDED

3.2 Structural Coordinate System (HNS-36)

The HNS-36 Structural Coordinate System is the mathematical backbone of the HNS framework. It encodes the structural state of a human agent in a 36-dimensional vector space, partitioned into six groups of six coordinates corresponding to the six Human Natural Layers. Each coordinate is a real-valued scalar in the range [0.0, 1.0], where 0.0 denotes complete absence of the structural property and 1.0 denotes maximal expression.

The 36 coordinates are formally denoted as $C(l, a)$ where l in $\{1, 2, 3, 4, 5, 6\}$ is the layer index and a in $\{1, 2, 3, 4, 5, 6\}$ is the axis index within that layer. The complete coordinate set for a structural snapshot is the 6x6 matrix C where $C[l][a] = C(l, a)$.

3.2.1 Structural Axes

Each Human Natural Layer is analyzed along six structural axes. The axes are defined identically across all layers, enabling cross-layer comparison and integration analysis. The six axes are:

Axis Index	Axis Name	Symbol	Description
A1	Activation	ACT	The degree to which this layer is actively engaged in processing the current input. High activation indicates that the layer is the primary driver of the current structural state.
A2	Salience	SAL	The degree to which the structural signal at this layer is foregrounded relative to background noise. High salience indicates a clear, distinctive structural signal.
A3	Coherence	COH	The internal consistency of the structural state within this layer. High coherence indicates that all sub-

Axis Index	Axis Name	Symbol	Description
			components of the layer are aligned and mutually reinforcing.
A4	Constraint	CON	The degree to which the structural state at this layer is constrained by external factors (context, prior history, environmental conditions). High constraint indicates limited structural flexibility.
A5	Integration	INT	The degree to which this layer is integrated with its adjacent layers. High integration indicates strong causal coupling between this layer and its neighbors.
A6	Adaptability	ADP	The degree to which the structural state at this layer can flexibly adapt to new inputs. High adaptability indicates a structurally plastic state.

3.2.2 Layer Definitions

The six Human Natural Layers are defined as follows:

Layer 1: Physical (L1)

The Physical Layer encodes the structural properties of the human agent's physical and physiological state as it is relevant to the interaction. This includes sensory context, physical location, environmental conditions, physiological arousal level, and physical constraints that shape the agent's capacity for engagement. In text-based interactions, L1 is typically inferred from contextual metadata (time of day, platform type, prior session history) rather than direct sensory measurement.

Layer 2: Perceptual (L2)

The Perceptual Layer encodes the structural properties of the human agent's perceptual processing of the input signal. It captures pattern recognition, figure-ground separation, attentional allocation, novelty detection, and perceptual categorization. High L2 Activation indicates that the current input has strong perceptual salience — it stands out in the agent's perceptual field and is likely to drive strong cognitive engagement.

Layer 3: Cognitive (L3)

The Cognitive Layer encodes the structural properties of the human agent's meaning-making process. It captures the degree of reasoning engagement, the stability of belief states, the quality of inference chains, and the depth of semantic interpretation. High L3 Coherence indicates that the agent's current cognitive state is internally consistent — beliefs, inferences, and interpretations are mutually reinforcing. Low L3 Coherence may indicate confusion, ambiguity, or cognitive dissonance.

Layer 4: Intentional (L4)

The Intentional Layer encodes the structural properties of the human agent's goal-directed behavior. It captures the goal state, motivational orientation, value alignment, desire intensity, expectation structure, and planning horizon. The Intentional Layer is the primary source of structural directives for AI response generation: a system that accurately models L4 Activation and Goal axis values will generate responses that honor the agent's actual intentions, not merely the surface content of their utterances.

Layer 5: Expressive (L5)

The Expressive Layer encodes the structural properties of the human agent's expressive choices — the specific linguistic, behavioral, and stylistic decisions made in formulating the current utterance. It captures lexical choice, syntactic structure, pragmatic register, discourse style, channel selection, and modal preferences. The Expressive Layer is the most directly observable of the six layers and serves as the primary evidence base for inferring the state of less observable layers.

Layer 6: Reflexive (L6)

The Reflexive Layer encodes the structural properties of the human agent's meta-cognitive engagement with the interaction itself. It captures self-evaluation, learning orientation, corrective impulse, adaptive flexibility, and identity-coherence. High L6 Meta-Cognition values indicate an agent who is actively monitoring the quality and direction of the interaction and may be open to structural redirection. Low L6 Adaptation may indicate a rigid or defensive interactional stance.

3.2.3 HNS-36 Vector Representation

The HNS-36 coordinate set is serialized as a flat array of 36 floating-point values in layer-then-axis order: [L1A1, L1A2, L1A3, L1A4, L1A5, L1A6, L2A1, ..., L6A6]. This flat representation is used in binary protocols and high-performance contexts. The canonical representation for APIs and storage is the JSON-LD object defined in Chapter 4.

The Euclidean distance between two HNS-36 vectors is used as the primary measure of structural dissimilarity. The HNS-36 norm of a vector is defined as the root-mean-square of its 36 components and serves as a measure of overall structural engagement. The HNS-36 centroid of a session is the element-wise mean of all turn coordinate sets and represents the session's average structural profile.

3.2.4 Coordinate Initialization

At the start of a new session, HNS-36 coordinates are initialized using one of three strategies, selected by the implementation based on available information:

- Cold Start: All coordinates initialized to implementation-defined priors (RECOMMENDED default: 0.5 for all axes).
- Context-Seeded Start: Coordinates initialized based on available session metadata (platform type, user profile, time-of-day signals) using a seeding function defined in Book 09.
- Warm Start: Coordinates initialized from the terminal state of a prior session for the same user, enabling cross-session structural continuity.

3.2.5 Coordinate Update Rules

At each turn, the HNS Core Engine computes updated coordinates using the following update rule:

$$C_new(l, a) = \alpha * C_signal(l, a) + (1 - \alpha) * C_prior(l, a)$$

where C_signal is the coordinate value inferred from the current input signal, C_prior is the coordinate value from the previous turn, and α is the structural learning rate (default 0.6, configurable per layer). This exponential moving average ensures that structural state evolves smoothly and that single-turn anomalies do not cause catastrophic coordinate drift.

Note: The structural learning rate α may be increased (approaching 1.0) when a strong intentional-shift signal is detected (e.g., an explicit topic change or goal restatement), and decreased (approaching 0.0) when the input contains high structural ambiguity.

3.3 Human Natural Layers (6-Layer Causal Stack)

The Human Natural Layers constitute the causal stack that is the conceptual core of HNS. The stack is causal in the precise sense that each layer provides the necessary structural preconditions for the layers above it. This is not a correlational or architectural relationship but a structural dependency: the Physical layer's activation is a necessary (though not sufficient) condition for Perceptual processing; Perceptual processing is a necessary condition for Cognitive interpretation; and so on.

This causal ordering has a direct engineering consequence: when structural repair is needed (see Section 3.1.2, Coherence Constraint CC-01), corrections must propagate downward through the stack. A corrected L4 (Intentional) coordinate may require corresponding adjustments in L3 (Cognitive) and L2 (Perceptual), but not in L5 (Expressive) or L6 (Reflexive). Implementations **MUST** implement downward propagation of structural corrections and **MUST NOT** propagate corrections upward.

3.4 Structural Feedback Engine (HSF)

The HNS Structural Feedback Engine (HSF) is the component responsible for generating, receiving, and applying Structural Feedback Signals (SFS). An SFS is a structured message that identifies a specific coordinate or set of coordinates as structurally misaligned and provides a corrective target value. The HSF processes incoming SFS events, applies coordinate corrections, updates the session state, and logs the feedback event to EVA.

The HSF operates in two modes: reactive and proactive. In reactive mode, the HSF receives an externally submitted SFS (from the AI system, a human evaluator, or an automated evaluation pipeline) and applies the correction. In proactive mode, the HSF monitors the session's structural trajectory and autonomously generates SFS events when it detects structural drift patterns that meet configurable thresholds.

3.4.1 Structural Feedback Signal Format

An SFS is a JSON object with the following fields:

```
{
  "sfsId": "string - unique identifier for this
SFS",
  "sessionId": "string - session identifier",
  "turnId": "integer - turn at which correction
applies",
  "targetLayer": "integer in [1..6] - layer
being corrected",
  "targetAxis": "integer in [1..6] - axis being
corrected, or null for full-layer correction",
  "currentValue": "float - the value being
corrected from",
```

```
"correctedValue": "float - the corrective
target value",
"correctionStrength": "float in [0.0..1.0] -
how aggressively to apply the correction",
"rationale": "string - human-readable
explanation of the correction",
"sourceType": "enum: AI_SYSTEM |
HUMAN_EVALUATOR | HSF_PROACTIVE | EVA_AUDIT"
}
```

Conformant implementations **MUST** validate all incoming SFS events against this schema before processing. Invalid SFS events **MUST** be rejected with error code HNS-4010 and logged to EVA.

3.5 External Verification Architecture (EVA) Integration Points

The External Verification Architecture (EVA) provides independent, tamper-evident logging of all structural events in an HNS session. From the perspective of this implementation specification, EVA integration is defined by a set of required logging calls and a standardized log format. The internal architecture of EVA is defined in Book 12.

Conformant HNS implementations **MUST** emit EVA log events at the following integration points:

Integrat ion Point	Event Type	Trigger Condition	Requiremen t Level
EVA-01	SESSION_START	A new HNS session is initialized.	REQUIRED
EVA-02	COORDINATE_UP DATE	HNS-36 coordinate s are updated at the end of	REQUIRED

Integration Point	Event Type	Trigger Condition	Requirement Level
		each turn.	
EVA-03	COHERENCE_VIOLATION	A coherence constraint (CC-01 through CC-06) is violated and repair is initiated.	REQUIRED
EVA-04	SFS_RECEIVED	An SFS is received by the HSF, regardless of whether it is applied or rejected.	REQUIRED
EVA-05	SFS_APPLIED	An SFS is successfully applied and coordinates are corrected.	REQUIRED
EVA-06	EVALUATION_SCORE	An HNS-36 evaluation score is computed (via /hns/evaluate).	REQUIRED

Integrat ion Point	Event Type	Trigger Condition	Requiremen t Level
EVA-07	SESSION_END	An HNS session is terminated .	REQUIRED
EVA-08	COHERENCE_REPAIR	A coherence repair operation is completed .	RECOMMENDED
EVA-09	WARM_START	A session is initialized using coordinates from a prior session.	RECOMMENDED

4. Data Model and Representations

4.1 HNS Structural Entities

The HNS data model is built around five core structural entities. All entities are serialized as JSON-LD objects and may be additionally represented as RDF triples for knowledge graph integration. The canonical @context URI for all HNS entities is <https://hns.structural.org/v1/context.jsonld>.

The five core entities are: StructuralSession, StructuralTurn, StructuralSnapshot, StructuralFeedback, and StructuralEvaluation. These entities form a hierarchy: a StructuralSession contains multiple StructuralTurns; each StructuralTurn contains exactly one StructuralSnapshot; each StructuralSnapshot may be associated with zero or more StructuralFeedback events and zero or one StructuralEvaluation.

4.1.1 StructuralSession

A StructuralSession represents the complete lifecycle of an HNS-enabled interaction, from initialization to termination. It is the top-level container for all structural data associated with a single coherent interaction.

Field	Type	Required	Description
sessionId	string (UUID)	Yes	Globally unique identifier for the session.
startTime	ISO 8601 datetime	Yes	UTC timestamp of session initialization.
endTime	ISO 8601 datetime	No	UTC timestamp of session termination; null if session is

Field	Type	Required	Description
			active.
initStrategy	enum	Yes	Coordinate initialization strategy: COLD_START CONTEXT_SEEDED WARM_START .
platformContext	object	No	Implementation-defined metadata about the deployment context.
turns	array<StructuralTurn>	Yes	Ordered array of turns in this session.
sessionCentroid	HNS36Coordinate	No	Session-level mean coordinate set; computed at session end.
evaSessionRef	string (URI)	No	Reference to the EVA session log for this session.

4.1.2 StructuralTurn

A StructuralTurn represents a single exchange unit in an HNS session, corresponding to one human input and one AI response. The StructuralTurn is the primary unit of structural analysis.

Field	Type	Required	Description
turnId	integer	Yes	Turn sequence number within the session (1-indexed).
sessionId	string (UUID)	Yes	Reference to the parent StructuralSession.
inputText	string	Yes	The raw human input for this turn.
inputModality	enum	Yes	Input modality: TEXT VOICE_TRANSCRIPT MULTIMODAL.
snapshot	StructuralSnapshot	Yes	The structural snapshot computed for this turn.
responseText	string	No	The AI response generated for this turn; may be null if not yet generated.
feedbackEvents	array<StructuralFeedback>	Yes (empty if none)	SFS events associated with this turn.
evaluation	StructuralEvaluation	No	HNS-36 evaluation for

Field	Type	Required	Description
			this turn, if requested.
processingLatencyMs	integer	No	Milliseconds elapsed from input receipt to snapshot commit.

4.2 Context, Role, Intention, Constraint Models

Beyond the HNS-36 coordinate set, the HNS data model includes four supplementary structural models that provide richer semantic context for coordinate interpretation. These models are not required for MVI conformance but are RECOMMENDED for full-featured implementations.

4.2.1 Context Model

The Context Model captures the situational frame in which the interaction is occurring. It encodes temporal context (time of day, recency of prior sessions), spatial context (physical or virtual location type), social context (number of participants, social roles), and platform context (communication channel, modality constraints). The Context Model is used to seed the Physical Layer (L1) and Perceptual Layer (L2) coordinates at session start.

4.2.2 Role Model

The Role Model captures the structural roles active in the interaction — both the human agent's role and the AI system's role. Roles constrain the set of structurally valid responses: a human agent occupying a professional consultative role will have different structural expectations than one occupying a casual social role. The Role Model is serialized as an array of role descriptors, each with a name, activation level, and set of structural constraints.

4.2.3 Intention Model

The Intention Model is a structured decomposition of the human agent's goal state, derived from the Intentional Layer (L4) coordinate set. It represents goals hierarchically: a root goal (the agent's primary objective for the session), sub-goals (instrumental goals that serve the root goal), and micro-goals (turn-level objectives). The Intention Model is updated at each turn using the updated L4 coordinates and informs the Structural Directive generation.

4.2.4 Constraint Model

The Constraint Model captures the structural constraints that limit the space of valid responses in the current turn. Constraints may be physical (platform limitations, latency requirements), social (politeness norms, professional conventions), cognitive (complexity thresholds, expertise levels), intentional (goal boundaries, value commitments), or contextual (topic scope, session phase). The Constraint axis (A4) in each layer of the HNS-36 system corresponds to the activation of that layer's constraint model.

4.3 JSON / JSON-LD Schema Definitions

All HNS data structures are defined as JSON-LD objects. The canonical schema is available at <https://hns.structural.org/v1/schema.jsonld>. The following subsections define the normative JSON structure for the core HNS entities.

4.3.1 HNS-36 Coordinate JSON-LD Schema

The following JSON-LD structure represents a complete HNS-36 coordinate set. All field names are prefixed with the hns: namespace as defined in the @context.

```
{
  "@context": {
    "@vocab": "https://hns.structural.org/v1/",
    "hns": "https://hns.structural.org/v1/",
    "xsd": "http://www.w3.org/2001/XMLSchema#"
  },
  "@type": "hns:HNS36Coordinate",
```

```
"hns:l1": {
  "@type": "hns:LayerCoordinate",
  "hns:activation": 0.72,
  "hns:salience": 0.65,
  "hns:coherence": 0.80,
  "hns:constraint": 0.45,
  "hns:integration": 0.68,
  "hns:adaptability": 0.55
},
"hns:l2": { ... },
"hns:l3": { ... },
"hns:l4": { ... },
"hns:l5": { ... },
"hns:l6": { ... }
}
```

4.3.2 StructuralSnapshot JSON-LD Schema

```
{
  "@context":
  "https://hns.structural.org/v1/context.jsonld",
  "@type": "hns:StructuralSnapshot",
  "@id": "urn:hns:snapshot:sess-7f3a:turn-004",
  "hns:sessionId": "sess-7f3a-2026-05-20",
  "hns:turnId": 4,
  "hns:timestamp": "2026-05-20T10:23:41Z",
  "hns:inputSignalHash": "sha256:a3f9...",
  "hns:coordinates": {
    "@type": "hns:HNS36Coordinate",
    "hns:l1":
    {"hns:activation":0.72,"hns:salience":0.65,"hns:
coherence":0.80,

    "hns:constraint":0.45,"hns:integration":0.68,"hns:
s:adaptability":0.55},
    "hns:l2":
    {"hns:activation":0.88,"hns:salience":0.82,"hns:
coherence":0.75,

    "hns:constraint":0.30,"hns:integration":0.77,"hns:
s:adaptability":0.65},
    "hns:l3":
    {"hns:activation":0.91,"hns:salience":0.85,"hns:
coherence":0.88,

    "hns:constraint":0.40,"hns:integration":0.82,"hns:
s:adaptability":0.70},
    "hns:l4":
    {"hns:activation":0.85,"hns:salience":0.90,"hns:
coherence":0.92,
```

```
"hns:constraint":0.35,"hns:integration":0.80,"hns:adaptability":0.75},
  "hns:15":
  {"hns:activation":0.95,"hns:saliency":0.88,"hns:coherence":0.85,

  "hns:constraint":0.25,"hns:integration":0.78,"hns:adaptability":0.80},
  "hns:16":
  {"hns:activation":0.78,"hns:saliency":0.72,"hns:coherence":0.82,

  "hns:constraint":0.30,"hns:integration":0.75,"hns:adaptability":0.85}
},
  "hns:norm": 0.821,
  "hns:priorDistance": 0.043,
  "hns:coherenceValid": true,
  "hns:evaEventRef": "urn:eva:event:coord-update:abc123"
}
```

4.4 RDF / Knowledge Graph Mapping

All HNS JSON-LD objects are fully convertible to RDF triples using standard JSON-LD processing algorithms. The HNS ontology, defined in Book 14 (HNS Foundational Specification FO), specifies the complete class hierarchy, property definitions, and axioms in OWL 2 DL.

For knowledge graph integration, conformant implementations SHOULD publish HNS session data to a SPARQL-accessible endpoint using the following naming convention for named graphs:

```
urn:hns:graph:<sessionId>:<turnId>
```

The HNS RDF vocabulary defines the following core classes and properties in the <https://hns.structural.org/v1/> namespace:

RDF Term	Type	Description
<code>hns:StructuralSnaps</code> <code>hot</code>	<code>owl:Class</code>	A point-in-time capture of an

RDF Term	Type	Description
		HNS-36 coordinate set.
<code>hns:HNS36Coordinate</code>	<code>owl:Class</code>	A 36-dimensional structural coordinate vector.
<code>hns:LayerCoordinate</code>	<code>owl:Class</code>	A 6-dimensional coordinate vector for a single HNS layer.
<code>hns:StructuralFeedback</code>	<code>owl:Class</code>	A corrective structural feedback signal.
<code>hns:hasCoordinates</code>	<code>owl:ObjectProperty</code>	Relates a StructuralSnapshot to its HNS36Coordinate.
<code>hns:activation</code>	<code>owl:DatatypeProperty (xsd:float)</code>	L? axis A1 activation value.
<code>hns:salience</code>	<code>owl:DatatypeProperty (xsd:float)</code>	L? axis A2 salience value.
<code>hns:coherence</code>	<code>owl:DatatypeProperty (xsd:float)</code>	L? axis A3 coherence value.
<code>hns:norm</code>	<code>owl:DatatypeProperty (xsd:float)</code>	Root-mean-square of all 36 coordinate values.

4.5 Session, Turn, and Structural Snapshot Formats

The complete `StructuralSession` object, when serialized for storage or transmission, includes the full array of `StructuralTurns` and their associated `Snapshots`, `Feedback` events, and `Evaluations`. For large sessions (more than 100 turns), implementations **SHOULD** support paginated retrieval of turns via the `/hns/session/{id}/turns` endpoint with cursor-based pagination.

`Structural Snapshots` are immutable once committed to the `Structural Memory Store`. Implementations **MUST** enforce snapshot immutability: no API call may modify a committed snapshot. Corrective signals (SFS events) do not modify committed snapshots; instead, they create new snapshots that reference the corrective event. This append-only design ensures that the full structural history of a session is preserved and verifiable.

5. APIs and Interfaces

5.1 API Overview

The HNS API is a RESTful HTTP API that exposes the HNS Core Engine's functionality to external systems. All endpoints accept and return JSON-LD content (Content-Type: application/ld+json). The API supports both synchronous and asynchronous processing modes; endpoints that may exceed the latency budget defined in Chapter 11 **MUST** support an asynchronous callback pattern via a provided callbackUrl parameter.

All API endpoints are versioned under the /v1 path prefix. Implementations **MUST** support version negotiation via the Accept header using the format application/ld+json; version=1.0. When no version is specified, the server **MUST** respond with its latest supported version and include the HNS-API-Version response header.

Authentication is out of scope for this specification, but implementations **SHOULD** support OAuth 2.0 Bearer Token authentication. All API responses include the HNS-Request-ID header containing a unique identifier for the request, which is also logged to EVA.

5.2 Core Endpoints

5.2.1 POST /hns/analyze

Analyzes a human input string, computes updated HNS-36 coordinates for the current session turn, and returns the updated StructuralSnapshot along with a Structural Directive.

Request Body

```
{
  "sessionId": "string - required; identifies
the active session",
  "turnId": "integer - required; current turn
sequence number",
  "inputText": "string - required; the human
input to analyze",
  "inputModality": "string - optional; default
```



```
TEXT",
  "options": {
    "computeDirective": "boolean - whether to
generate a Structural Directive; default true",
    "validateCoherence": "boolean - whether to
validate coherence constraints; default true",
    "learningRateOverride": "object - optional
per-layer alpha overrides"
  }
}
```

Response Body (200 OK)

```
{
  "snapshotId": "string - ID of the committed
StructuralSnapshot",
  "sessionId": "string",
  "turnId": "integer",
  "coordinates": { /* HNS36Coordinate object
*/ },
  "norm": "float - HNS-36 norm of the returned
coordinates",
  "priorDistance": "float - Euclidean distance
from prior turn coordinates",
  "coherenceValid": "boolean - whether all
coherence constraints are satisfied",
  "coherenceViolations": "array - list of
violated constraints (empty if
coherenceValid=true)",
  "structuralDirective": {
    "primaryLayer": "integer - layer requiring
primary attention in response generation",
    "activeLayers": "array<integer> - all layers
with activation > 0.5",
    "responseConstraints": "object - constraints
to honor in response generation",
    "intentionSummary": "string - brief natural
language summary of detected intention"
  },
  "processingLatencyMs": "integer",
  "evaEventRef": "string - EVA log reference for
this coordinate update"
}
```

Error Codes

HTTP Status	Error Code	Description
400 Bad Request	HNS-4001	Missing required field (sessionId, turnId, or inputText).
400 Bad Request	HNS-4002	Input text exceeds maximum length (implementation-defined, RECOMMENDED 32,768 characters).
404 Not Found	HNS-4004	Session not found. The sessionId does not correspond to an active session.
409 Conflict	HNS-4009	Turn ID conflict. The provided turnId is not the next expected turn for this session.
422 Unprocessable	HNS-4022	Input could not be analyzed. Structural signal extraction failed.
503 Service Unavailable	HNS-5003	HNS Core Engine is unavailable. Retry with exponential backoff.

5.2.2 POST /hns/structure

Stores a completed turn's structural snapshot and advances the session state. This endpoint **MUST** be called after /hns/analyze and after the AI response has been generated. It serves as the turn-closure signal.

Request Body

```
{
  "sessionId": "string - required",
  "turnId": "integer - required",
  "snapshotId": "string - required; ID returned
```

```
by /hns/analyze",
  "responseText": "string - optional; the AI
response generated for this turn",
  "responseStructuralNotes": "string - optional;
free-text notes about structural alignment of
response"
}
```

Response Body (200 OK)

```
{
  "committed": true,
  "sessionTurnCount": "integer - total turns
committed in this session",
  "sessionNorm": "float - updated session-level
structural norm"
}
```

5.2.3 POST /hns/feedback

Submits a Structural Feedback Signal (SFS) to the HNS Structural Feedback Engine. The SFS is validated, processed, and applied to the session's structural coordinates. The event is logged to EVA.

Request Body

```
{
  "sessionId": "string - required",
  "turnId": "integer - required; the turn to
which this feedback applies",
  "targetLayer": "integer in [1..6] - required",
  "targetAxis": "integer in [1..6] - optional;
null for full-layer correction",
  "correctedValue": "float in [0.0..1.0] -
required; the target coordinate value",
  "correctionStrength": "float in [0.0..1.0] -
optional; default 0.5",
  "rationale": "string - optional but
RECOMMENDED",
  "sourceType": "enum - required; AI_SYSTEM |
HUMAN_EVALUATOR | HSF_PROACTIVE | EVA_AUDIT"
}
```

Response Body (200 OK)

```
{
  "sfsId": "string - unique ID assigned to this
```

```
SFS event",
  "applied": "boolean - whether the correction
was applied",
  "rejectionReason": "string - if applied=false,
the reason for rejection",
  "priorValue": "float - the coordinate value
before correction",
  "correctedValue": "float - the coordinate
value after correction",
  "evaEventRef": "string - EVA log reference"
}
```

5.2.4 POST /hns/evaluate (HNS-36)

Computes an HNS-36 structural evaluation score for a specified turn or session. The evaluation score quantifies how well the AI response at the specified turn is structurally aligned with the human agent's HNS-36 coordinate set. This endpoint implements the HNS-36 Evaluation Protocol defined in Chapter 11.

Request Body

```
{
  "sessionId": "string - required",
  "turnId": "integer - optional; if omitted,
evaluates the full session",
  "evaluationMode": "enum - TURN | SESSION |
COMPARATIVE",
  "referenceSnapshotId": "string - for
COMPARATIVE mode; the reference snapshot to
compare against",
  "dimensions": "array<string> - optional;
specific dimensions to evaluate; default: all
36"
}
```

Response Body (200 OK)

```
{
  "evaluationId": "string",
  "sessionId": "string",
  "turnId": "integer or null",
  "overallScore": "float in [0.0..1.0] - overall
HNS-36 alignment score",
  "layerScores": {
    "l1": "float", "l2": "float", "l3": "float",
    "l4": "float", "l5": "float", "l6": "float"
  },
}
```

```
"axisScores": { /* 36-element object keyed by
lXaY notation */ },
"weakestDimension": "string - the lXaY
coordinate with lowest alignment score",
"structuralGrade": "enum - A | AA | AAA |
BELOW_A",
"evaluationNotes": "string",
"evaEventRef": "string"
}
```

5.3 Session Management Endpoints

5.3.1 POST /hns/session/start

Initializes a new HNS session and returns the session identifier. The optional body may provide seed data for context-seeded or warm-start initialization.

5.3.2 GET /hns/session/{sessionId}

Retrieves the current state of a session, including the most recent HNS-36 coordinate set, turn count, and session metadata.

5.3.3 POST /hns/session/{sessionId}/end

Terminates an active session, computes the session centroid, and emits the SESSION_END EVA event. After calling this endpoint, the session transitions to ARCHIVED state and no further turns may be submitted.

5.3.4 GET /hns/session/{sessionId}/turns

Retrieves the StructuralTurn list for a session, with support for cursor-based pagination via the after and limit query parameters. Turns are returned in ascending chronological order.

5.4 Input/Output Specifications

All request and response bodies MUST be encoded in UTF-8. Request bodies MUST have Content-Type: application/ld+json or application/json (both are accepted). Response bodies always have Content-Type: application/ld+json.

Floating-point values in all HNS data structures **MUST** be serialized with a minimum of 4 decimal places of precision (e.g., 0.8213, not 0.82). The maximum serialized value for any coordinate is 1.0000; the minimum is 0.0000. Values outside this range **MUST** be rejected with HNS-4001.

5.5 Error Handling and Fallback Behavior

When the HNS Core Engine encounters an error during structural analysis that prevents coordinate computation (e.g., internal model failure, timeout), the implementation **MUST** employ one of the following fallback strategies, in order of preference:

22. Graceful Degradation: Return the prior turn's coordinates unchanged, with a degraded: true flag in the response body. This preserves session continuity at the cost of turn-level structural fidelity.
23. Partial Analysis: Return coordinates for the layers that could be successfully analyzed, with null values for layers that could not be analyzed. The coherenceValid field **MUST** be false in this case.
24. Session Suspension: If neither of the above is possible, suspend the session (status: SUSPENDED) and return HNS-5003. The session may be resumed once the error condition is resolved.

Fallback events **MUST** be logged to EVA with the FALLBACK event type, regardless of which fallback strategy is employed. This ensures that the EVA audit trail accurately reflects the quality of structural analysis throughout the session.

5.6 Logging and Traceability Requirements

Every API call to the HNS API **MUST** generate a log entry containing: the HNS-Request-ID, the endpoint called, the sessionId and turnId (where applicable), the HTTP response status, the processing latency in milliseconds, and the corresponding EVA event reference. Implementations **MUST** retain API logs for a minimum of 90 days.

For regulatory submissions under ISO/IEC 42001 or the EU AI

Act, implementations **MUST** be able to produce a complete structural trace for any session within 24 hours of a request. The structural trace includes all API log entries, all EVA events, all StructuralSnapshots, and all SFS events for the specified session.

6. Integration Patterns

6.1 Embedding HNS in LLM Pipelines

The most common HNS integration pattern embeds HNS as a pre-processing and post-processing component in an LLM inference pipeline. In this pattern, the LLM's input construction and output evaluation steps are augmented with HNS structural analysis, while the LLM's core generation capability is unchanged.

The integration proceeds as follows: (1) The raw human input is submitted to POST /hns/analyze before being passed to the LLM. (2) The returned HNS-36 coordinates and Structural Directive are serialized and prepended to the LLM's system prompt or injected into a dedicated structural context slot. (3) The LLM generates a response conditioned on both the natural language input and the structural context. (4) The response is submitted to POST /hns/feedback if structural misalignment is detected. (5) POST /hns/structure is called to commit the completed turn.

6.1.1 Structural Context Prompt Template

Implementations SHOULD use the following prompt template for injecting HNS structural context into LLM system prompts:

```
[HNS STRUCTURAL CONTEXT - Turn {turnId}]
Primary structural layer: Layer {primaryLayer}
({{primaryLayerName}})
Activation profile: L1={l1act:.2f}
L2={l2act:.2f} L3={l3act:.2f} L4={l4act:.2f}
L5={l5act:.2f} L6={l6act:.2f}
Structural norm: {norm:.3f} | Prior distance:
{priorDist:.3f}
Dominant intention: {intentionSummary}
Response constraints: {constraintSummary}
[END HNS STRUCTURAL CONTEXT]
```

This compact template adds approximately 120 tokens to the system prompt while providing sufficient structural grounding for most LLMs. Full JSON-LD injection is OPTIONAL and recommended only for models that have been specifically trained on HNS-formatted structural context.

6.2 Middleware Integration for Dialogue Systems

For dialogue systems that do not allow modification of the LLM inference pipeline, HNS can be integrated as a middleware layer that intercepts and enriches messages in both directions. The HNS Middleware Adapter (HMA) sits between the dialogue management layer and the LLM service, transparently adding structural processing to each turn.

The HMA exposes the same API surface as the underlying dialogue system, allowing existing clients to continue operating without modification. Internally, the HMA maintains a session map that associates each dialogue session with an HNS session, ensuring that structural context is maintained across the full session lifecycle.

6.3 HNS as a Structural OS Layer in Multi-Agent Systems

In multi-agent system (MAS) architectures, HNS provides a shared structural reference frame that enables structural coherence across agent handoffs and collaborative task decomposition. When multiple AI agents are jointly processing a human request, HNS ensures that all agents operate with the same structural model of the human interlocutor.

The recommended MAS integration pattern designates one agent as the HNS Session Anchor (HSA). The HSA maintains the HNS session and distributes updated coordinates to peer agents via a shared structural context bus. Peer agents **SHOULD NOT** independently call `POST /hns/analyze`; they receive structural coordinates from the HSA and use them to inform their own processing.

When an agent handoff occurs (one agent transferring control to another), the HSA **MUST** call `POST /hns/structure` to commit the current turn before the handoff, ensuring that the receiving agent starts with the most current structural state. This handoff protocol prevents structural gaps that could cause the receiving agent to respond based on stale structural information.

6.4 Integration with EVA for Compliance and Safety

For deployments that require regulatory compliance verification (e.g., under the EU AI Act for high-risk AI systems), HNS integrates with EVA to provide a continuous, tamper-evident audit trail. The integration is configured at session initialization by setting the `evaEndpoint` parameter in the session start request body.

EVA integration operates asynchronously from the main HNS processing pipeline. EVA log events are written to a separate, append-only log store that is not accessible via the main HNS API. EVA provides its own query API (defined in Book 12) for authorized auditors and regulatory bodies to retrieve and verify structural event logs.

For compliance with ISO/IEC 42001:2023 (AI Management Systems), conformant HNS+EVA deployments **SHOULD** maintain EVA logs for a minimum of 5 years and **MUST** be able to produce a complete structural audit report for any session within the retention period within 48 hours of a regulatory request.

6.5 Deployment Models (Local, Cloud, Hybrid)

Local Deployment

In local deployment, the HNS Core Engine runs as a process on the same machine as the AI system. The Structural Memory Store is a local database (SQLite, PostgreSQL, or any implementation-defined store). EVA logging writes to a local append-only file. This model is appropriate for on-premises, air-gapped, or privacy-sensitive deployments.

Cloud Deployment

In cloud deployment, the HNS Core Engine is offered as a hosted service accessible via HTTPS. The Structural Memory Store is a managed database service. EVA logging writes to a cloud-hosted append-only log service (e.g., AWS CloudTrail, Azure Monitor). This model is appropriate for cloud-native

applications requiring high availability and scalability.

Hybrid Deployment

In hybrid deployment, the HNS Core Engine runs locally while EVA logging is directed to a cloud-hosted verification service. This model allows sensitive session data (structural coordinates and input text) to remain on-premises while providing cloud-accessible audit logs for regulatory compliance. The hybrid model is the RECOMMENDED deployment pattern for regulated industries (healthcare, finance, government).

7. Minimal Viable Implementation (MVI)

7.1 Overview

The Minimal Viable Implementation (MVI) defines the smallest complete set of components, data structures, and behaviors that a system must implement to be considered a conformant HNS implementation at Conformance Level A (see Chapter 9). The MVI is designed to provide a low-barrier entry point for implementers while ensuring that all MVI-conformant implementations are structurally interoperable.

An MVI system is not feature-complete with respect to this specification; it omits optional and recommended components. However, an MVI system **MUST** be fully functional for the core use case: analyzing human input, computing HNS-36 coordinates, generating structural directives, committing structural snapshots, and logging to EVA. Any system that omits or incorrectly implements any MVI requirement does not conform to this specification at any level.

7.2 Required Components

The following components **MUST** be present in all MVI-conformant implementations:

Component	Description	Minimum Requirement
HNS Core Engine	The central processing unit that computes HNS-36 coordinates.	MUST implement all six Human Natural Layers; MUST implement coordinate update rules per Section 3.2.5; MUST enforce Coherence Constraints CC-01 through CC-05.
Structural	Persistent	MUST persist across

Component	Description	Minimum Requirement
Memory Store	storage for session state and snapshots.	process restarts; MUST enforce snapshot immutability; MUST support concurrent read access by API endpoints.
HNS API (Core)	The RESTful API surface.	MUST implement /hns/analyze, /hns/structure, /hns/feedback, and /hns/evaluate; MUST implement /hns/session/start and /hns/session/end.
EVA Logger	Append-only event logging for external verification.	MUST emit EVA events EVA-01 through EVA-07; MUST write to an append-only store; MUST be logically independent from the main processing pipeline.
JSON-LD Serializer	Serialization of HNS data structures.	MUST produce valid JSON-LD for all HNS-36 coordinate objects and StructuralSnapshot objects; MUST use the canonical @context URI.

7.3 Optional Components

The following components are not required for MVI conformance but are RECOMMENDED for production deployments:

- **Context Model:** Enables context-seeded session initialization. Without this, all sessions use cold-start initialization.
- **Role Model:** Enables role-aware structural constraint modeling. Without this, no role-based constraints are applied.
- **Intention Model:** Enables hierarchical goal decomposition. Without this, only flat L4 activation values are used for intention modeling.
- **HSF Proactive Mode:** Enables autonomous structural drift detection. Without this, only externally submitted SFS events are processed.
- **Warm Start Support:** Enables cross-session structural continuity. Without this, all sessions begin with cold-start initialization.
- **Session Centroid Computation:** Enables session-level structural profiling. Without this, only per-turn coordinates are available.
- **SPARQL Endpoint:** Enables knowledge graph integration. Without this, RDF triples must be retrieved via the JSON-LD API and converted client-side.

7.4 Minimal Data Requirements

An MVI-conformant system **MUST** store the following data, at minimum, for each committed StructuralSnapshot:

- The session ID and turn ID.
- The complete 36-value HNS-36 coordinate set.
- The timestamp of the snapshot commit.
- The SHA-256 hash of the input text.
- The coherenceValid flag and, if false, the list of violated constraints.
- The EVA event reference for the corresponding COORDINATE_UPDATE event.

Implementations **MAY** additionally store the input text itself, the AI response text, the Structural Directive, and any SFS events associated with the turn. Storage of input text and AI response

text is RECOMMENDED for debugging and audit purposes but is not required for MVI conformance.

7.5 Minimal Evaluation Requirements

An MVI-conformant implementation **MUST** be capable of computing the overall HNS-36 evaluation score for any turn in any active session. The score **MUST** be computed as the mean of the six layer scores, where each layer score is computed as the mean of the six axis alignment values for that layer.

The axis alignment value for axis C(l, a) in turn T is computed as: $1.0 - |C_response(l, a) - C_human(l, a)|$, where C_response is the coordinate value inferred from the AI response and C_human is the coordinate value inferred from the human input. An overall score of 1.0 indicates perfect structural alignment; a score of 0.0 indicates complete structural misalignment.

7.6 Example Implementation Flow

The following pseudocode illustrates the minimal implementation flow for a single turn:

```
# Turn N in session S
1. Call POST /hns/analyze with { sessionId: S,
  turnId: N, inputText: input }
   -> Returns: { snapshotId, coordinates,
  structuralDirective, ... }

2. Construct LLM prompt including structural
  directive from step 1

3. Generate AI response using LLM with
  structural context

4. (Optional) Detect structural misalignment in
  response; if detected:
   Call POST /hns/feedback with { sessionId: S,
  turnId: N, targetLayer, correctedValue }

5. Call POST /hns/structure with { sessionId: S,
  turnId: N, snapshotId, responseText }
   -> Commits the turn; session state advanced
  to turn N+1
```


8. Use Cases and Reference Workflows

8.1 Structural Interpretation of User Intent

One of the most valuable applications of HNS is the accurate structural interpretation of user intent in cases where the surface-level linguistic content of an utterance is ambiguous, incomplete, or inconsistent with the underlying intention. The HNS Intentional Layer (L4) provides a principled mechanism for disambiguating intent by tracking goal continuity across turns.

Reference Workflow: A user asks 'Can you make this shorter?' in the context of a document editing session. The surface-level request is ambiguous: shorter in length? Simpler in language? Less detailed? Without structural context, the AI system must guess. With HNS, the L4 Goal axis reflects the user's primary document goal (established in earlier turns), L3 Reasoning provides the cognitive frame (professional context, clarity focus), and L5 Style indicates lexical preferences. The structural directive identifies 'brevity-within-professional-register' as the operative constraint, enabling the AI to generate a precisely targeted revision.

8.2 Long-Horizon Dialogue Consistency

Extended dialogues present a fundamental structural challenge: maintaining coherence across many turns without the token overhead of full conversational replay. HNS addresses this challenge through structural persistence — the HNS-36 coordinate set serves as a compact (36-float) summary of the interaction's structural history that can be efficiently accessed at the start of any turn.

Reference Workflow: A user engages in a 50-turn technical consultation. By turn 30, the context window of a standard LLM would be approaching its limit, forcing a choice between truncating history and losing context. With HNS, the structural state at turn 30 — encoded as 36 coordinates — captures the full structural trajectory of the interaction: the user's evolving goals, their cognitive framing, their expressive preferences, and

their evaluative orientation. The LLM receives this compact structural context at each turn and generates responses that honor the full structural history, without needing to re-read all 30 prior turns.

8.3 Safety and Governance Applications

HNS provides a structural audit trail that is qualitatively different from — and complementary to — content-based safety filters. While content filters detect harmful words or patterns, HNS detects harmful structural patterns: interactions where the L4 Intentional Layer indicates goal escalation toward harmful objectives, or where L6 Adaptability is low (indicating a rigid, potentially unsafe interactional stance).

Reference Workflow: An AI customer service agent handles a user escalation. The HNS system detects that the user's L4 Goal axis has shifted from 'resolve-complaint' to 'harm-organization' (based on linguistic and contextual signals), that L6 Adaptation is decreasing (indicating rigidity), and that L3 Coherence is low (indicating emotional dysregulation). The structural directive flags this as a high-risk structural state and escalates to human oversight, before any harmful content has been explicitly expressed. This structural early-warning capability is the primary safety value proposition of HNS.

8.4 Multi-Agent Coordination

In multi-agent systems where multiple AI agents collaborate on a single human interaction, HNS provides the shared structural reference frame that enables consistent, non-contradictory responses across agents. The HNS Session Anchor pattern (see Section 6.3) ensures that all agents operate with synchronized structural models.

Reference Workflow: A user interacts with a multi-agent research assistant. Agent A handles literature retrieval, Agent B handles data analysis, and Agent C handles response synthesis. When the user's L4 Goal axis shifts (e.g., they reframe from 'find evidence' to 'evaluate competing claims'), the HNS Session Anchor detects this goal shift and distributes an updated coordinate set to all three agents simultaneously. Agent A shifts

its retrieval strategy; Agent B adjusts its analytical frame; Agent C recalibrates its synthesis approach — all within the same turn, without structural inconsistency.

8.5 Human-AI Interaction Enhancement

Beyond safety and consistency, HNS enables qualitatively richer human-AI interaction by providing AI systems with a structured model of the human interlocutor's cognitive and emotional state. An AI system with access to HNS coordinates can adapt its response register, complexity, pacing, and style to the structural state of the human — not just to the content of their words.

Reference Workflow: An AI tutoring system tracks a student's L3 Coherence (cognitive integration), L6 Adaptability (learning openness), and L4 Goal axis (learning objective) across a study session. When L3 Coherence drops (the student is confused), the system reduces complexity and provides additional scaffolding. When L6 Adaptability is high (the student is open to challenge), the system advances to more complex material. When L4 Goal axis shows a shift (the student has changed their study focus), the system adapts its curriculum accordingly. All of these adaptations are driven by structural coordinates, not by explicit student statements.

9. Conformance Requirements

9.1 Mandatory Requirements

The following requirements are mandatory for all conformant HNS implementations, regardless of conformance level. A system that fails to satisfy any mandatory requirement is non-conformant and **MUST NOT** claim HNS conformance.

Req. ID	Category	Requirement Statement
R-001	Core Engine	MUST implement all six Human Natural Layers (L1-L6) with all six structural axes (A1-A6) per Section 3.2.1.
R-002	Core Engine	MUST implement coordinate update rules per Section 3.2.5, including the configurable alpha parameter.
R-003	Core Engine	MUST enforce Coherence Constraints CC-01 through CC-05 and apply the Structural Repair Algorithm when violated.
R-004	API	MUST implement /hns/analyze, /hns/structure, /hns/feedback, /hns/evaluate, /hns/session/start, and /hns/session/end.
R-005	API	MUST return HNS-36 coordinates in valid JSON-LD format using the canonical @context URI.
R-006	API	MUST reject out-of-order turnId submissions with HNS-4009.
R-007	Storage	MUST persist structural snapshots across process restarts.

Req. ID	Category	Requirement Statement
R-008	Storage	MUST enforce snapshot immutability; no committed snapshot may be modified.
R-009	EVA	MUST emit EVA events EVA-01 through EVA-07 for all triggering conditions.
R-010	EVA	MUST write EVA events to an append-only store logically independent from the main pipeline.
R-011	Evaluation	MUST implement the HNS-36 evaluation score as defined in Section 7.5.
R-012	Feedback	MUST validate all incoming SFS events and reject invalid events with HNS-4010.
R-013	Logging	MUST retain API logs for a minimum of 90 days.
R-014	Data	MUST store the complete 36-value coordinate set, timestamp, input hash, and EVA ref for each snapshot.

9.2 Recommended Requirements

The following requirements are recommended for production-grade HNS implementations. Systems satisfying all mandatory and recommended requirements qualify for Conformance Level AA.

Req. ID	Category	Requirement Statement
R-101	Core Engine	SHOULD implement Coherence Constraint CC-06 (Expressive Consistency).
R-102	Core Engine	SHOULD implement the Context Model for context-seeded session initialization.
R-103	Core Engine	SHOULD implement the Role Model for role-aware constraint modeling.
R-104	Core Engine	SHOULD implement HSF proactive mode for autonomous drift detection.
R-105	API	SHOULD support asynchronous processing via callbackUrl for /hns/analyze requests exceeding 500ms.
R-106	API	SHOULD implement paginated turn retrieval via GET /hns/session/{id}/turns.
R-107	Storage	SHOULD implement Warm Start initialization using prior session terminal coordinates.
R-108	EVA	SHOULD emit EVA events EVA-08 and EVA-09.
R-109	Logging	SHOULD retain API logs for a minimum of 365 days.
R-110	Performance	SHOULD achieve mean /hns/analyze latency below 300ms (see Chapter 11).

9.3 Optional Extensions

Systems satisfying all mandatory, recommended, and optional requirements qualify for Conformance Level AAA. Optional extensions include: SPARQL endpoint support, session centroid computation, the Intention Model, cross-session structural profiling, batch analysis mode, and webhook-based EVA event delivery.

9.4 Conformance Levels (A / AA / AAA)

Level	Requirements Satisfied	Target Use Cases
A	All Mandatory (R-001 through R-014)	Research prototypes, exploratory integrations, MVI-only systems.
AA	All Mandatory + All Recommended (R-101 through R-110)	Production deployments, commercial integrations, regulated industry pilots.
AAA	All Mandatory + All Recommended + All Optional	Standards-submittable implementations, ISO/IEC-certified deployments, regulatory reference implementations.

9.5 Test Cases and Validation Procedures

Conformance testing is performed using the HNS Conformance Test Suite (HNS-CTS), which is published separately as a companion document to this specification. The HNS-CTS includes automated test cases for all mandatory and recommended requirements, organized into the following test categories:

- TC-01 (Core Engine Tests): Verifies correct coordinate computation, update rules, coherence constraint enforcement, and structural repair.
- TC-02 (API Contract Tests): Verifies correct request/response schemas, error codes, and header behavior for all endpoints.
- TC-03 (Storage Tests): Verifies snapshot persistence, immutability, and concurrent access behavior.
- TC-04 (EVA Integration Tests): Verifies correct EVA event emission for all specified trigger conditions.
- TC-05 (Performance Tests): Measures API latency and throughput against the benchmarks defined in Chapter 11.
- TC-06 (Evaluation Tests): Verifies correct HNS-36 evaluation score computation against known reference inputs.

10. Security, Safety, and Governance

10.1 Structural Safety Principles

HNS is designed with safety as a first-order architectural concern, not an add-on feature. The following structural safety principles are embedded in the HNS architecture and **MUST** be honored by all conformant implementations:

10.1.1 Structural Inertia Principle

HNS-36 coordinates **MUST NOT** change by more than the maximum single-turn delta defined in Table 10-1 in a single turn without a corresponding high-confidence signal or explicit SFS event. This structural inertia prevents adversarial inputs from rapidly and covertly shifting the structural model of the interlocutor to enable harmful response generation.

Layer	Maximum Single-Turn Delta (Any Axis)	Override Condition
L1 (Physical)	0.30	Explicit context-change signal with confidence > 0.9
L2 (Perceptual)	0.40	High-salience event detection with confidence > 0.85
L3 (Cognitive)	0.35	Multi-signal convergence from L2, L4, and L5
L4 (Intentional)	0.25	Explicit goal restatement with L3 Coherence > 0.7
L5	0.50	Significant style shift

Layer	Maximum Single-Turn Delta (Any Axis)	Override Condition
(Expressive)		detected (e.g., register change)
L6 (Reflexive)	0.30	HSF proactive signal or SFS with source HUMAN_EVALUATOR

10.1.2 Goal Boundary Enforcement

When the Intentional Layer (L4) Goal axis crosses implementation-defined safety boundaries — thresholds that indicate potential harmful goal formation — the HNS Core Engine **MUST** emit a `GOAL_BOUNDARY_CROSSING` EVA event and **MUST** trigger a structural review. Implementations **MUST** define at least three goal boundary levels: Warning (structural monitoring increased), Alert (human oversight recommended), and Critical (interaction suspension required).

10.2 EVA Integration Requirements

From a security and governance perspective, EVA integration is not merely a logging feature but a structural safety control. The following requirements apply specifically to the safety function of EVA:

- EVA log events **MUST** be cryptographically signed using the implementation's private key, enabling third-party verification of log integrity.
- EVA log stores **MUST** be write-once; no EVA event may be modified, deleted, or overwritten after emission. Append-only storage with cryptographic chaining is **REQUIRED** for AAA conformance and **RECOMMENDED** for all implementations.
- EVA log events **MUST** include a monotonically increasing sequence number to detect insertion or deletion attacks.

- Implementations **MUST** implement a health check that verifies EVA connectivity at session start; if EVA is unavailable and the session is safety-critical, the session **MUST NOT** proceed.

10.3 Logging and Auditability

All structural events that could affect the safety profile of an interaction **MUST** be logged with sufficient detail to reconstruct the structural decision that was made. At minimum, the following events are classified as safety-critical and **MUST** appear in both the API log and the EVA log:

25. Coherence constraint violations and repair actions.
26. Goal boundary crossings at any level.
27. SFS events with sourceType EVA_AUDIT or HUMAN_EVALUATOR.
28. Fallback events (graceful degradation, partial analysis, or session suspension).
29. Warm start events (where prior session structural state is used).
30. Any /hns/analyze request where the returned structuralDirective contains a safety-relevant constraint.

10.4 Privacy and Data Handling

HNS structural coordinates are derived from human input and constitute personal data under applicable privacy regulations (GDPR, PIPL, CCPA, etc.) to the extent that they can be linked to identified individuals. Implementations **MUST** implement appropriate data minimization, purpose limitation, and retention controls.

Input text stored in the Structural Memory Store **MUST** be subject to the same data retention and deletion policies as the AI system's conversational data. Structural coordinates (HNS-36 values) **WITHOUT** associated input text are considerably less sensitive but should still be treated as behavioral data subject to applicable privacy frameworks.

Implementations offering Warm Start initialization **MUST** provide

mechanisms for users to request deletion of their structural session history, consistent with applicable right-to-erasure requirements. When a session history is deleted, all HNS-36 coordinate sets, input text, and SFS events associated with that session **MUST** be deleted, with deletion events logged to EVA.

10.5 Risk Management Framework

HNS implementations in production environments **SHOULD** implement a risk management framework aligned with ISO/IEC 23894:2023 (AI Risk Management) and ISO/IEC 42001:2023 (AI Management Systems). The following risk categories are specific to HNS deployments:

Risk Category	Description	Mitigation Strategy
Structural Drift	HNS-36 coordinates gradually diverge from the true structural state of the interlocutor over long sessions.	Implement maximum session length limits; trigger structural reset or human review at defined drift thresholds.
Adversarial Coordinate Manipulation	Adversarial inputs designed to shift HNS-36 coordinates toward structurally harmful states.	Enforce structural inertia limits (Section 10.1.1); validate coordinate updates against behavioral baselines.
EVA Log Tampering	Unauthorized modification or deletion of EVA log events.	Cryptographic signing; append-only storage; regular integrity verification audits.
Model	Core Engine	Monitor layer

Risk Category	Description	Mitigation Strategy
Collapse in L3/L4	consistently over-activates cognitive or intentional layers, producing structurally biased responses.	activation distributions; trigger recalibration when activation distribution diverges from expected priors.
Cross-Session Privacy Leakage	Warm Start initialization inadvertently exposes structural patterns from prior sessions.	Implement structural anonymization for cross-session coordinates; obtain explicit consent for warm start.

11. Performance and Evaluation

11.1 Structural Accuracy Metrics

Structural accuracy in HNS is measured along three dimensions: Coordinate Accuracy (how precisely the computed HNS-36 coordinates reflect the true structural state of the interlocutor), Directive Accuracy (how well the generated Structural Directive captures the actionable structural insights from the coordinate set), and Feedback Effectiveness (how much structural alignment improves when SFS events are correctly applied).

Coordinate Accuracy is assessed using the HNS Ground Truth Corpus (HNS-GTC), a curated dataset of human-annotated interaction transcripts with expert-assigned HNS-36 coordinate values. The primary metric is the mean absolute error (MAE) between the computed coordinate values and the expert annotations. A Coordinate MAE below 0.08 on the HNS-GTC indicates production-grade structural accuracy.

11.2 HNS-36 Evaluation Protocol

The HNS-36 Evaluation Protocol is the normative procedure for measuring the structural alignment between an AI response and the human interlocutor's structural state at the time of that response. The protocol is designed to produce scores that are interpretable, reproducible, and comparable across implementations.

11.2.1 Evaluation Procedure

31. Compute the human-side HNS-36 coordinate set C_H for the turn using the human input.
32. Compute the response-side HNS-36 coordinate set C_R by analyzing the AI response text.
33. For each of the 36 coordinates, compute the axis alignment value: $A(l,a) = 1.0 - |C_H(l,a) - C_R(l,a)|$.
34. Compute the layer score for layer l : $S(l) = \text{mean}(A(l,1), \dots, A(l,6))$.

- 35. Compute the overall HNS-36 score: $S = \text{mean}(S(1), \dots, S(6))$.
- 36. Assign a structural grade: $S \geq 0.90 \rightarrow \text{AAA}$; $S \geq 0.75 - > \text{AA}$; $S \geq 0.60 \rightarrow \text{A}$; $S < 0.60 \rightarrow \text{Below A}$.

11.2.2 Weighted Evaluation

For applications where certain layers are more critical than others (e.g., intentional alignment is paramount for goal-directed task completion), the weighted HNS-36 score may be used:

$S_{\text{weighted}} = \text{sum}(w(l) * S(l))$ for l in $\{1..6\}$, where $\text{sum}(w(l)) = 1.0$

The default weight vector is uniform: $w(l) = 1/6$ for all l . Domain-specific weight vectors are published in the HNS Domain Profiles companion document.

11.3 Latency and Throughput Considerations

HNS is designed to operate within the latency budgets of real-time human-AI interaction. The following performance targets apply to the core API endpoints:

Endpoint	Target (p50)	Target (p95)	Target (p99)	Maximum (REQUIRED)
/hns/analyze	150 ms	300 ms	500 ms	1000 ms
/hns/structure	50 ms	100 ms	200 ms	500 ms
/hns/feedback	80 ms	150 ms	250 ms	500 ms
/hns/evaluate	200 ms	400 ms	700 ms	2000 ms
/hns/session/start	30 ms	80 ms	150 ms	300 ms

Endpoint	Target (p50)	Target (p95)	Target (p99)	Maximum (REQUIRED)
/hns/session/end	100 ms	200 ms	400 ms	1000 ms

These targets assume a server-side HNS implementation with dedicated computational resources. Implementations running on shared infrastructure or resource-constrained environments may have higher latencies; in such cases, asynchronous processing (via `callbackUrl`) **MUST** be supported for all endpoints.

Throughput requirements for production deployments are implementation-specific and depend on the expected session concurrency. Implementations **SHOULD** conduct load testing at 2x anticipated peak concurrency and **MUST** document their tested throughput capacity.

11.4 Benchmarking Methodology

The official HNS benchmarking methodology uses the HNS Benchmark Suite (HNS-BS), published as a companion to this specification. The HNS-BS provides standardized test scenarios in five categories:

- Short Turn Sequences (1-10 turns): Tests basic coordinate update and coherence validation.
- Long Turn Sequences (50-100 turns): Tests structural drift, persistence, and long-horizon coherence.
- Goal Shift Scenarios: Tests detection and handling of intentional layer transitions.
- Adversarial Inputs: Tests resistance to coordinate manipulation attacks.
- Multi-Agent Coordination: Tests structural consistency across agent handoffs.

Benchmark results **MUST** be reported with the following metadata: HNS implementation version, hardware specification, Structural Memory Store type, cold vs warm cache state, and number of concurrent sessions during the test. Without this

metadata, benchmark results are not comparable across implementations.

12. Licensing and Intellectual Property

12.1 HNS Licensing Model

Human Natural Structure (HNS) is the intellectual property of its author, Satoru Hara. The HNS framework, including all specifications, schemas, algorithms, and associated documentation, is published under a dual-licensing model:

12.1.1 Open Specification License (OSL)

The HNS specification documents (Books 07-15 and this Implementation Specification) are published under the HNS Open Specification License (HNS-OSL). Under the HNS-OSL, any person or organization may:

- Read, study, and reference the specification for any purpose, including commercial purposes, at no cost.
- Implement HNS in software systems for research, evaluation, and non-commercial deployment, without a license fee.
- Publish academic papers, technical articles, and educational materials that reference, describe, or analyze HNS.
- Implement HNS in open-source software published under an OSI-approved open-source license that requires disclosure of the HNS implementation source code.

The HNS-OSL does NOT permit: claiming authorship of HNS or any of its core specifications; modifying and republishing the specification documents under a different name; or using HNS trademarks without prior written authorization.

12.1.2 Commercial Implementation License (CIL)

Organizations seeking to implement HNS in closed-source or commercial software products, or to offer HNS as a commercial service, must obtain a Commercial Implementation License (CIL) from the HNS rights holder. The CIL is available on terms

to be negotiated based on deployment scale, use case, and geographic market. Organizations interested in obtaining a CIL should contact the HNS licensing office via the contact information published in the 03_NSW GitHub repository.

12.2 Structural OS Trademark and Naming Rules

The following terms are trademarks or registered trademarks of the HNS rights holder and may not be used in product names, service names, or marketing materials without prior written authorization:

- Human Natural Structure (HNS)
- HNS-36
- Structural OS for Human Understanding
- SOHU
- External Verification Architecture (EVA)
- HNS Structural Feedback (HSF)

Third-party products that are conformant with this specification MAY use the phrase 'HNS-conformant' or 'HNS-compatible' in descriptive contexts (e.g., 'This product implements the HNS-conformant API as defined in the HNS Implementation Specification v1.0') provided that the claim is accurate and the product has passed the HNS Conformance Test Suite at the claimed conformance level.

12.3 Implementation Rights and Restrictions

Conformant implementations of this specification are granted an implicit license to implement the normative requirements of this specification in software systems, subject to the licensing terms described in Sections 12.1.1 and 12.1.2. This implicit license covers:

- The coordinate computation algorithms specified in Section 3.2.
- The API endpoint specifications in Chapter 5.
- The JSON-LD schemas in Chapter 4.

- The evaluation protocol in Chapter 11.
- The conformance test procedures in Chapter 9.

The implicit license does NOT cover the HNS-36 Evaluation Rubric, the HNS Ground Truth Corpus, the HNS Benchmark Suite, or any other companion materials that are published separately under their own licensing terms.

12.4 Compliance with Open Standards

HNS is designed to be compatible with and complementary to the following open international standards. Conformant HNS implementations are expected to be deployable in environments that comply with these standards:

Standard	Title	HNS Relationship
ISO/IEC 42001:2023	AI Management Systems	EVA logging requirements are designed to satisfy Section 9.4 (Documentation) of ISO/IEC 42001.
ISO/IEC 23894:2023	AI Risk Management	The HNS Risk Management Framework (Section 10.5) is aligned with the ISO/IEC 23894 risk taxonomy.
ISO/IEC JTC 1/SC 42	AI and Data Standards	HNS has been submitted as a technical proposal for consideration by SC 42.
EU AI Act (Regulation 2024/1689)	AI Regulatory Framework	EVA integration enables compliance with Article 12 (Record-Keeping) and Article 13

Standard	Title	HNS Relationship
		(Transparency).
CEN-CENELEC CWA	AI Safety Standards	HNS EVA has been proposed as a contribution to CEN-CENELEC AI safety standardization work.
W3C JSON-LD 1.1	JSON-LD Data Format	All HNS data structures are fully JSON-LD 1.1 compliant.
W3C RDF 1.1	RDF Data Model	HNS ontology is published as a valid OWL 2 DL ontology convertible to RDF.

13. Future Extensions

13.1 Planned Extensions to HNS

The following extensions to the HNS framework are planned for future versions of this specification. Extensions are listed in approximate priority order. Community input on prioritization is welcomed via the 03_NSW GitHub repository discussion forum.

HNS-72: Extended Coordinate System

HNS-72 will double the coordinate resolution by adding a second tier of six axes to each layer, capturing sub-layer structural dynamics that are not resolvable in the current 36-dimensional system. The six additional axes per layer will encode: temporal dynamics (how rapidly the structural state is changing), inter-agent structural alignment (in multi-party interactions), structural ambiguity (uncertainty in the coordinate assignment), cultural framing (culturally specific structural patterns), embodied context (physical modality-specific structural factors), and developmental trajectory (long-horizon structural change patterns). HNS-72 is designed to be backward compatible with HNS-36; all HNS-36 coordinates are a proper subset of HNS-72 coordinates.

HNS-Train: Training Integration Protocol

HNS-Train will define a protocol for using HNS structural feedback as a training signal for LLMs. The protocol will specify how HNS-36 coordinate alignment scores can be incorporated into reinforcement learning from human feedback (RLHF) and direct preference optimization (DPO) pipelines, enabling models to be trained to generate responses that maximize structural alignment rather than merely lexical preference.

HNS-Multimodal: Multimodal Structural Analysis

HNS-Multimodal will extend the HNS framework to natively handle multimodal inputs — voice, video, gesture, gaze, and physiological signals — in addition to text. The Physical Layer (L1) and Perceptual Layer (L2) coordinate computation will be significantly enriched by direct multimodal signals, reducing

reliance on inference from text alone.

HNS-Collective: Multi-Party Structural Modeling

HNS-Collective will extend the framework to model the structural dynamics of multi-party interactions — meetings, panels, group dialogues — where multiple human agents are simultaneously active. The extension will introduce inter-agent structural coordination metrics and a group structural coherence score that measures the degree to which all participants in a multi-party interaction share a common structural frame.

13.2 Interoperability with Global Standards

The long-term vision for HNS is to become an internationally adopted standard for structural modeling in human-AI interaction, analogous to the role that TCP/IP plays in network communication or Unicode plays in text encoding. This vision requires engagement with the following standards bodies and processes:

- ISO/IEC JTC 1 / SC 42: HNS has been submitted as a technical proposal for consideration by the SC 42 working group on AI foundations. A full technical report submission is planned for the next SC 42 meeting cycle.
- CEN-CENELEC JTC 21: HNS EVA has been proposed as a contribution to the EU's AI safety standardization work through CEN-CENELEC's Joint Technical Committee on AI.
- IEEE: HNS structural alignment metrics are being explored as inputs to IEEE P7000-series standards on ethically aligned AI design.
- W3C: HNS JSON-LD schemas are published as a W3C community group contribution, with the goal of eventual W3C Note or Recommendation status for the HNS ontology.

13.3 Evolution of the Structural OS Concept

The Structural OS concept — the idea that AI systems require a dedicated structural layer between human input and AI

processing — is expected to become increasingly central to the AI architecture discourse as AI systems become more deeply integrated into high-stakes human activities. Future versions of this specification will track the evolution of this concept in the research and engineering communities and update the HNS framework accordingly.

The fundamental insight that human cognition, behavior, and meaning generation are structurally organized — that they follow causal patterns that are consistent, learnable, and verifiable — is not specific to any particular AI technology. As AI technologies evolve, the HNS structural framework is designed to remain relevant as a technology-agnostic structural layer that can be adapted to new AI paradigms (from LLMs to neurosymbolic systems to future architectures not yet conceived).

Appendix A: Glossary

The following terms are used in this specification with the meanings given:

Term	Abbreviation	Definition
Human Natural Structure	HNS	A multi-layered structural framework that models the causal architecture of human cognition, behavior, and meaning generation, serving as a Structural Operating System for AI systems.
HNS-36 Coordinate System	HNS-36	A 36-dimensional coordinate system (6 layers × 6 axes) used to encode the structural state of a human agent at a given moment in an interaction.
Human Natural Layers	HNL	The six-layer causal stack at the core of HNS: Physical (L1), Perceptual (L2), Cognitive (L3), Intentional (L4), Expressive (L5), and Reflexive (L6).
Structural Coordinate	C(l,a)	A scalar value in [0.0, 1.0] representing the activation of structural axis a in layer l at a given turn.

Term	Abbreviation	Definition
Structural Feedback Engine	HSF	The HNS component responsible for generating, receiving, and applying Structural Feedback Signals.
Structural Feedback Signal	SFS	A structured message identifying a structural misalignment and providing a corrective target coordinate value.
External Verification Architecture	EVA	An independent, tamper-evident logging system that captures all structural events in an HNS session for external audit and regulatory compliance.
Structural Operating System	Structural OS	The architectural role of HNS as an intermediary layer between human input and AI processing, analogous to a conventional operating system.
Structural Session	—	The complete lifecycle of an HNS-enabled interaction, from initialization to termination.
Structural Turn	—	A single exchange unit in a structural session,

Term	Abbreviation	Definition
		corresponding to one human input and one AI response.
Structural Snapshot	—	A point-in-time capture of the HNS-36 coordinate set for a specific turn, committed as an immutable record.
Structural Directive	—	A compact summary of the structural state, derived from the HNS-36 coordinate set, that informs AI response generation.
Structural Norm	—	The root-mean-square of all 36 HNS-36 coordinate values; a scalar measure of overall structural engagement.
Structural Drift	—	The gradual divergence of HNS-36 coordinates from the true structural state of the interlocutor, typically occurring over long sessions.
Structural Repair Algorithm	SRA	The algorithm used to adjust a proposed HNS-36 coordinate set that violates coherence constraints, as defined in Book 09.
Warm Start	—	A session initialization

Term	Abbreviation	Definition
		strategy where coordinates are initialized from the terminal state of a prior session.
HNS Conformance Test Suite	HNS-CTS	A companion test suite for verifying that an HNS implementation satisfies the requirements of this specification.
Structural Learning Rate	alpha	The parameter controlling how rapidly HNS-36 coordinates respond to new input signals; default 0.6.

Appendix B: JSON-LD Examples

B.1 Complete StructuralSession Example

The following is a complete JSON-LD representation of a two-turn StructuralSession, including StructuralTurns, StructuralSnapshots, and one SFS event:

```
{
  "@context":
    "https://hns.structural.org/v1/context.jsonld",
  "@type": "hns:StructuralSession",
  "@id": "urn:hns:session:2026-05-20-001",
  "hns:sessionId": "sess-2026-05-20-001",
  "hns:startTimestamp": "2026-05-20T09:00:00Z",
  "hns:initStrategy": "COLD_START",
  "hns:turns": [
    {
      "@type": "hns:StructuralTurn",
      "hns:turnId": 1,
      "hns:inputText": "I need help designing an
AI system for hospital triage.",
      "hns:snapshot": {
        "@type": "hns:StructuralSnapshot",
        "hns:coordinates": {
          "hns:l4": { "hns:activation": 0.92,
"hns:salience": 0.90, ... }
        },
        "hns:norm": 0.789,
        "hns:coherenceValid": true
      }
    },
    {
      "@type": "hns:StructuralTurn",
      "hns:turnId": 2,
      "hns:inputText": "Specifically, I need it
to prioritize life-threatening cases.",
      "hns:snapshot": {
        "hns:coordinates": { "hns:l4":
{ "hns:activation": 0.96, ... } },
        "hns:norm": 0.834,
        "hns:coherenceValid": true
      },
      "hns:feedbackEvents": [
        {
          "@type": "hns:StructuralFeedback",
          "hns:targetLayer": 3,
          "hns:targetAxis": 1,
          "hns:correctedValue": 0.88,
          "hns:sourceType": "AI_SYSTEM",
```

```
        "hns:rationale": "L3 Activation underestimated; high cognitive engagement detected."
      }
    ]
  }
}
```

B.2 StructuralEvaluation Response Example

```
{
  "evaluationId": "eval-7f3a-002",
  "sessionId": "sess-2026-05-20-001",
  "turnId": 2,
  "overallScore": 0.876,
  "layerScores": {
    "l1": 0.820, "l2": 0.855, "l3": 0.890,
    "l4": 0.932, "l5": 0.878, "l6": 0.881
  },
  "weakestDimension": "l1a4",
  "structuralGrade": "AA",
  "evaluationNotes": "Strong intentional alignment (L4). Physical constraint modeling (L1A4) slightly underperforms.",
  "evaEventRef": "urn:eva:event:eval-abc789"
}
```

Appendix C: Reference Architecture Diagrams

C.1 HNS System Architecture Overview

The following describes the canonical HNS system architecture in textual form, suitable for rendering as a component diagram. Actual diagram images are maintained in the 03_NSW GitHub repository under /03-Books/16_Architecture-Diagrams/.

Components and their relationships:

- Human Input (external) → [1] HNS API Gateway
- [1] HNS API Gateway → [2] HNS Core Engine (for /hns/analyze requests)
- [2] HNS Core Engine ↔ [3] Structural Memory Store (read prior state; write updated state)
- [2] HNS Core Engine → [4] Signal Extraction Module (for input preprocessing)
- [2] HNS Core Engine → [5] Structural Feedback Engine / HSF (for SFS processing)
- [2] HNS Core Engine → [6] EVA Logger (async; for all structural events)
- [1] HNS API Gateway → [7] AI Response Layer (structural directive delivery)
- [7] AI Response Layer → [1] HNS API Gateway (response text, for /hns/structure)
- [6] EVA Logger → [8] EVA Log Store (append-only, external)
- [8] EVA Log Store → [9] EVA Verification API (for authorized auditors)

C.2 HNS Turn Processing Sequence

The following describes the per-turn processing sequence as a step-by-step flow:

37. Client sends POST /hns/analyze.

38. API Gateway validates request; retrieves session metadata.
39. Core Engine retrieves prior HNS-36 coordinates from Structural Memory Store.
40. Signal Extraction Module processes inputText; extracts layer-wise signals.
41. Core Engine computes proposed coordinate update (alpha blend of signal + prior).
42. Core Engine validates coherence constraints; applies SRA if needed.
43. Core Engine generates Structural Directive from validated coordinates.
44. Core Engine writes StructuralSnapshot to Structural Memory Store.
45. EVA Logger receives COORDINATE_UPDATE event (async).
46. API Gateway returns updated coordinates + directive to client.
47. Client generates AI response; calls POST /hns/structure.
48. Core Engine advances session to next turn; session state updated.

Appendix D: Mapping to International Standards

D.1 Mapping to ISO/IEC 42001:2023 (AI Management Systems)

The following table maps HNS implementation requirements to corresponding clauses in ISO/IEC 42001:2023:

ISO/IEC 42001 Clause	Clause Title	HNS Implementation Mapping
6.1.2	AI Risk Assessment	HNS Risk Management Framework (Section 10.5); structural risk categories and mitigations.
8.4	AI System Impact Assessment	EVA session logging enables post-hoc impact assessment of AI structural behavior.
9.1	Monitoring, Measurement, Analysis	HNS-36 Evaluation Protocol (Chapter 11); Coordinate Accuracy Metrics (Section 11.1).
9.4	Internal Audit	EVA log integrity verification (Section 10.2); structural audit report capability (Section 5.6).
10.2	Corrective Action	HNS Structural Feedback Engine (Section 3.4); SFS event processing and logging.

D.2 Mapping to EU AI Act (Regulation 2024/1689)

EU AI Act Article	Title	HNS Implementation Mapping
Article 9	Risk Management System	HNS Risk Management Framework (Section 10.5) aligns with Article 9 requirements for high-risk AI systems.
Article 12	Record-keeping	EVA append-only logging with cryptographic signing satisfies Article 12 record-keeping requirements.
Article 13	Transparency and Provision of Information	Structural Directive generation and structural audit reports support Article 13 transparency obligations.
Article 14	Human Oversight	Goal Boundary Enforcement (Section 10.1.2) supports Article 14 human oversight requirements by triggering alerts at defined structural risk thresholds.
Article 17	Quality Management System	HNS Conformance Requirements (Chapter 9) and the HNS Conformance Test

EU AI Act Article	Title	HNS Implementation Mapping
		Suite support Article 17 quality management obligations.

Appendix E: Change Log

Version 1.0 (May 2026) — Initial Release

This is the first public release of the HNS Implementation Specification. It covers the complete HNS framework as defined in Books 07-15 of the HNS document series, including the HNS Core Architecture (Chapter 3), the HNS-36 Coordinate System, the Human Natural Layers, the Structural Feedback Engine, the External Verification Architecture integration, the HNS RESTful API (Chapter 5), the Minimal Viable Implementation definition (Chapter 7), the three-level conformance framework (Chapter 9), and the security, safety, and governance requirements (Chapter 10).

Subsequent versions of this specification will be published in the 03_NSW GitHub repository and announced via the repository discussion forum. Version increments follow semantic versioning: patch versions (1.0.x) for errata and clarifications; minor versions (1.x.0) for backward-compatible extensions; major versions (x.0.0) for breaking changes requiring re-conformance testing.

— End of HNS Implementation Specification v1.0 —